README

Transaction Ledger Management System

Transaction Ledger Management System

A full-stack ledger application for creating accounts, recording immutable credits and debits, reviewing balances and history, and making atomic transfers between accounts.

Features

- Account creation with a server-generated UUID and a protected zero starting balance.
- Exact-decimal credit/debit posting, per-account idempotency references, and insufficient- funds checks.
- Atomic money transfers with paired debit/credit entries.
- Account balances, a server-aggregated dashboard, and searchable, paginated transaction history.
- Responsive React dashboard, account management, forms, transaction search, and API-connected cash-flow chart.
- Django REST Framework API, automated tests, MySQL configuration, SQLite quick-start mode.

Technology

- React 18, Vite, React Router, Recharts, Framer Motion, Lucide icons, custom responsive CSS.
- Python 3.10+, Django 5, Django REST Framework, PyMySQL.
- MySQL 8+ recommended for deployed use. SQLite is the test/development default.

Project structure

```
transaction-ledger/
??? backend/
?   ??? config/                # Django settings and URL routing
?   ??? ledger/                # Models, serializers, API views, tests, migrations
?   ??? .env.example
?   ??? manage.py
?   ??? requirements.txt
??? frontend/
?   ??? src/
?   ?   ??? components/        # Layout and shared UI
?   ?   ??? pages/             # Dashboard, accounts, transaction flows
?   ?   ??? services/          # Central API client
?   ?   ??? styles.css
?   ??? .env.example
?   ??? package.json
??? API_DOCUMENTATION.md
??? ASSUMPTIONS.md
??? .gitignore
```

Prerequisites

- Python 3.10 or later (3.12 recommended).
- Node.js 20 LTS or later and npm.
- MySQL 8+ for MySQL mode. For the fastest local startup, use SQLite mode and skip the MySQL setup.

Backend setup

Run these commands from the project root in PowerShell. Reuse the root .venv if it is already present; create it only for a fresh checkout:

```
Set-Location D:\hema # For a
fresh checkout only:
py -3 -m venv .venv
# Reuse the existing environment when it is already present:
.\.venv\Scripts\Activate.ps1 python -m pip install --upgrade
pip pip install -r backend\requirements.txt Copy-Item
backend\.env.example backend\.env
Set-Location backend
```

If backend\.env already exists, do not overwrite it when copying the example. Relative SQLite database paths are resolved from backend/, so the example uses the existing backend\db.sqlite3 file.

If PowerShell blocks activation, allow it for the current terminal only:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
..\.venv\Scripts\Activate.ps1
```

The checked-in example uses SQLite. Django loads backend/.env; for MySQL, edit it and set DBENGINE=mysql plus the connection settings below. The .env file is local-only and ignored by Git. SQLite is suitable for local development and tests; use MySQL/InnoDB when concurrent posting must rely on row-level locks.

MySQL configuration (optional local setup)

Create a database and a dedicated local user using a MySQL client with administrator access:

```
CREATE DATABASE transaction_ledger CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'ledger_user'@'localhost' IDENTIFIED BY 'use-a-strong-local-password';
GRANT ALL PRIVILEGES ON transaction_ledger.* TO 'ledger_user'@'localhost';
FLUSH PRIVILEGES;
```

Set these entries in backend/.env:

```
DB_ENGINE=mysql
DB_NAME=transaction_ledger
DB_USER=ledger_user
DB_PASSWORD=your-local-password
DB_HOST=127.0.0.1
DB_PORT=3306
```

PyMySQL is used as the MySQL driver to keep the setup portable on Windows. Use InnoDB tables for transaction and row-lock guarantees.

Migrations, tests, and server

From backend/ with the root virtual environment active:

```
python manage.py migrate python manage.py
check python manage.py test ledger python
manage.py runserver 127.0.0.1:8000
```

The root URL http://127.0.0.1:8000/ and API root http://127.0.0.1:8000/api/ return a JSON index of the available routes. Django tests use a separate test database and do not reset or delete the configured SQLite database.

Frontend setup

In another terminal, from the project root:

```
Set-Location D:\hema\frontend
npm install Copy-Item
.env.example .env npm run dev
```

Open the Vite URL shown in the terminal (normally http://localhost:5173). Set VITEAPIBASEURL in frontend/.env if the API runs at a different address. The backend's CORSALLOWEDORIGINS must include the frontend origin.

Create a production frontend bundle with:

```
npm run build
npm run preview
```

API testing

See APIDOCUMENTATION.md (./APIDOCUMENTATION.md) for all endpoints, fields, response envelopes, and status codes. For a quick manual check after starting Django:

```
curl.exe -X POST http://127.0.0.1:8000/api/accounts/ `
-H "Content-Type: application/json" `
  -d '{"holder_name":"Mira Patel"}'
```

Create a credit by replacing ACCOUNTUUID with the returned UUID:

```
curl.exe -X POST http://127.0.0.1:8000/api/accounts/ACCOUNT_UUID/transactions/ `
-H "Content-Type: application/json" `
  -d '{"type":"credit","amount":"125.00","reference_id":"INV-2025-001","description":"Invoice payment"}'
```

Automated backend tests:

```
cd backend
..\.venv\Scripts\Activate.ps1
python manage.py test ledger
```

The test suite covers account validation and zero balances, credits/debits, insufficient funds, exact precision and type validation, duplicate references, cross-account reference reuse, balances, immutability, filtering/pagination, newest-first ordering, and atomic transfers. SQLite does not provide SELECT ... FOR UPDATE; validate concurrent posting behavior on MySQL/InnoDB before deployment.

Troubleshooting

- ModuleNotFoundError in Django: activate the root virtual environment and rerun pip install -r backend\requirements.txt.
- MySQL connection refused: confirm the MySQL service is running, the port/credentials are correct, and the database exists. Switch DBENGINE=sqlite for local work that does not require MySQL; relative SQLite database paths resolve from backend/.
- Browser CORS error: add the exact Vite origin (usually http://localhost:5173 or http://127.0.0.1:5173) to CORSALLOWEDORIGINS in backend/.env, then restart Django.
- Frontend cannot reach API: confirm Django is serving on port 8000 and VITEAPIBASEURL includes /api; restart Vite after changing .env.
- Debit/transfer rejected: check the available balance; debits cannot overdraw an account.
- Duplicate reference response: a repeated reference on the same account is intentionally treated as a successful idempotent retry and does not post again.
- Windows PowerShell activation is blocked: use the process-scoped execution policy command in Backend setup; it does not permanently change system policy.

Security and production notes

Authentication is intentionally not included because it was not specified in the assessment. This app is a local assessment project: do not expose it publicly without adding authentication/authorization, HTTPS, production secret management, appropriate host settings, backups, and monitoring. Never commit .env credentials.

Related documentation

- APIDOCUMENTATION.md (./APIDOCUMENTATION.md)
- ASSUMPTIONS.md (./ASSUMPTIONS.md)